INGENIERÍA DE SOFTWARE · CURSO 2026-2027

Seminario de Arquitecturas Distribuidas

Estilos de arquitectura para sistemas distribuidos

Equipo MERXIRVE

Integrante	Grupo	Pregunta / inciso
Enrique González González	C311	Pregunta 3; Pregunta 5
Ernesto Javier Govea Varona	C311	Pregunta 6
Olivia Ortiz Arboláez	C311	Pregunta 2
Anthony Cruz Garcia	C311	Pregunta 4
Jose Manuel Porras Guerra	C311	Pregunta 1

Repositorio del proyecto

https://github.com/Anthonycg2003/sistema-horarios-docentes

Índice

1. Pregunta 1: Comunicación y acoplamiento en el cierre de una subasta 3
1.1. Acoplamiento dinámico de cada enfoque . . . 3
1.2. Fallo del pago después de notificar al ganador . . . 4

2. Pregunta 2: Elasticidad extrema en la venta de entradas 6
2.1. Arquitectura basada en espacio frente a microservicios . . . 6
2.2. Colisiones de datos en asientos numerados . . . 6

3. Pregunta 3: Fallos en una arquitectura dirigida por eventos 8
3.1. Impacto del fallo . . . 8
3.2. Patrón de eventos de flujo de trabajo . . . 8

4. Pregunta 4: Granularidad de los microservicios de recursos humanos 10
4.1. Error de fondo en el diseño . . . 10
4.2. Nueva descomposición . . . 10

5. Pregunta 5: Consistencia, despliegue y equipo en un sistema financiero 12
5.1. Consistencia de los datos . . . 12
5.2. Despliegue independiente . . . 12

6. Pregunta 6: Reutilización, de SOA a los microservicios 14
6.1. La reutilización como objetivo y como consecuencia . . . 14
6.2. Reutilización de lógica de negocio: la validación de clientes . . . 14
6.3. Reutilización operativa: logging y monitoreo . . . 15
6.4. Cuándo sigue justificándose la orquestación . . . 16
6.5. Un flujo que se beneficia de la orquestación explícita . . . 17

Referencias bibliográficas 19

Declaración de Uso de la Inteligencia Artificial 20

1. Pregunta 1: Comunicación y acoplamiento en el cierre de una subasta

1.1. Acoplamiento dinámico de cada enfoque

El cierre de una subasta obliga a notificar al ganador, cobrar y actualizar el inventario. Esas tres acciones pueden coordinarse con un mediador, con eventos, o quedar dentro de un solo servicio de dominio. El acoplamiento que cambia entre esas opciones es el dinámico: el que solo aparece en ejecución. La connascencia dinámica distingue la de ejecución (importa el orden en que corren los componentes), la de tiempo (deben estar disponibles a la vez), la de valor (varios datos tienen que cambiar de forma coherente) y la de identidad (deben referirse a la misma subasta y al mismo ganador) (Richards & Ford, 2020). En un sistema distribuido esas formas se combinan con tres decisiones: comunicación síncrona o asíncrona, consistencia atómica o eventual, y coordinación orquestada o coreografiada (Ford et al., 2021).

En la orquestación de microservicios un mediador posee el flujo y llama a notificación, pago e inventario. Los servicios de dominio no se conocen, así que entre ellos baja la connascencia de ejecución. Esa dependencia no se elimina: se concentra en el mediador, que es el único que conoce el orden, los contratos y el estado del cierre. Si las llamadas son síncronas, se suma connascencia de tiempo. El mediador y cada servicio tienen que estar disponibles en el mismo instante, y la latencia del cierre es la suma de las tres llamadas (Richards & Ford, 2020). El acoplamiento dinámico queda centralizado y es fácil de leer, al precio de que el mediador se convierte en el punto por el que pasa toda la operación.

En la coreografía nadie dirige el flujo. El componente que cierra la subasta publica un evento y los demás reaccionan. La comunicación asíncrona reduce la connascencia de tiempo: el servicio de pago puede estar caído y el mensaje espera en el broker. El coste se reparte. Cada consumidor depende del contrato del evento (connascencia estática de nombre y de significado) y el resultado de negocio depende de que las reacciones ocurran sobre la misma subasta y terminen en estados compatibles (connascencia de identidad y de valor). El orden deja de estar escrito en un solo componente y pasa a ser un protocolo implícito entre los participantes (Ford et al., 2021). Hay menos acoplamiento de disponibilidad y más acoplamiento de conocimiento del protocolo, distribuido y más difícil de observar.

En la arquitectura basada en servicios (*Service-Based Architecture*) las tres acciones viven en un servicio de dominio de grano grueso, en general con la base de datos de ese dominio (Richards & Ford, 2020). Notificar, cobrar y actualizar el inventario son pasos del mismo proceso, no llamadas remotas. Para este caso de uso casi desaparece la connascencia de tiempo y de ejecución entre componentes desplegados por separado. La connascencia de valor sigue existiendo (las tres acciones tienen que concordar), pero la resuelve una transacción local, que es la forma barata de ese acoplamiento. Lo que se acepta a cambio es un despliegue más grueso: los módulos del cierre cambian y se publican juntos. Este documento llama SBA a la arquitectura basada en espacio de la pregunta 2; aquí el estilo es el de servicios de dominio amplios, y por eso no uso esa sigla.

La tabla 1 junta el efecto sobre el acoplamiento con lo que ocurre cuando el pago falla.

Enfoque	Acoplamiento dinámico	Si el pago falla tras avisar
Orquestación de microservicios	Queda en el mediador. Sube la connascencia de tiempo si las llamadas son síncronas. Baja entre los servicios de dominio.	El mediador ve el fallo, reintenta y compensa. La consistencia es eventual. El aviso al ganador ya no entra en la misma transacción.
Coreografía de microservicios	Queda en el protocolo de eventos. Baja la connascencia de tiempo. Suben la de valor, la de identidad y la dependencia del contrato del evento.	Hace falta un evento de compensación. Hasta que se procesa, ganador, cobro e inventario no concuerdan, y nadie es dueño del flujo.
Servicio de dominio de grano grueso	El caso de uso no cruza la red. La connascencia de valor se resuelve dentro del servicio. El acoplamiento que queda es de despliegue.	Una transacción local puede abortar el cierre antes de confirmar el aviso. ACID es viable. Un defecto del pago afecta a todo el servicio.

Tabla 1: Acoplamiento dinámico y fallo del pago en el cierre de la subasta. Elaboración propia a partir de (Ford et al., 2021; Richards & Ford, 2020).

1.2. Fallo del pago después de notificar al ganador

El fallo llega cuando el ganador ya fue avisado y el cobro no se concretó. La pregunta práctica es si esas dos acciones pueden compartir una transacción atómica (ACID) o si el sistema solo puede reconciliarlas después (BASE).

Con orquestación el mediador se entera, porque él llamó al pago. Un *commit* distribuido de dos fases restauraría la atomicidad, pero ata la disponibilidad de los tres servicios al más lento y se evita en microservicios (Ford et al., 2021). Lo que sí encaja es una saga orquestada. El mediador guarda que la notificación ya se hizo, reintenta el cobro y, si el rechazo es definitivo, ordena las compensaciones: retirar o corregir el aviso y no descontar el inventario, o reponerlo si ya se descontó. La consistencia es eventual. La tolerancia a fallos del flujo es buena, porque hay un estado explícito y un solo lugar donde reintentar. El aviso al usuario es el paso delicado: si salió antes de saber que el pago iba a fallar, el ganador ya leyó un resultado que el dominio no puede garantizar. Por eso el mediador debería notificar solo después de un cobro confirmado, o enviar primero un aviso condicionado a ese cobro.

Con coreografía el evento de cierre ya fue consumido por la notificación. El fallo del pago no tiene un destinatario obligado. Hace falta publicar un evento de compensación, por ejemplo `PagoRechazado`, al que estén suscritos el inventario y la notificación. Mientras ese evento no se procesa, el sistema queda en un estado BASE: el ganador cree que se quedó con el lote, el inventario puede seguir apartado y el dinero no entró. Si el broker retiene el mensaje, el cobro se retoma cuando el servicio de pago vuelve, y esa parte de la tolerancia a fallos es alta. Si el servicio consumió el evento, falló a medias y no publicó la compensación, la subasta queda partida sin un componente responsable de detectarlo (Richards & Ford, 2020). La red perdona la caída del pago; la consistencia de negocio depende de que cada participante implemente la compensación y de que el contrato del evento la contemple.

En el servicio de grano grueso las tres acciones pueden entrar en una transacción local. Si el pago falla, la base de datos no confirma el inventario ni el cierre. ACID es realista porque hay un solo proceso y una sola base (Richards & Ford, 2020). El correo o la notificación externa no participan de ese *rollback* si se enviaron antes del *commit*. El cierre coherente con el estilo es persistir el resultado y emitir el aviso solo después de confirmar el cobro. La tolerancia a fallos es la del proceso único: el pago fallido aborta la operación entera y no queda un ganador avisado a medias. El coste es de aislamiento. Un defecto del módulo de pago puede indisponer el servicio de subastas completo, no solo el cobro.

Orquestación y coreografía separan el pago del resto y, por eso, resuelven este fallo con reintentos y

compensaciones, en consistencia eventual. El servicio grueso puede abortar el cierre dentro de una transacción local, siempre que el aviso al ganador salga después de confirmar el cobro. En los tres diseños, notificar antes de saber que el pago se concretó es lo que deja al sistema con un ganador anunciado y una subasta sin cerrar.

2. Pregunta 2: Elasticidad extrema en la venta de entradas

2.1. Arquitectura basada en espacio frente a microservicios

En un escenario de venta masiva de entradas, la principal limitación no es el cálculo del sistema, sino la base de datos compartida. Cuando pasan de cientos a decenas de miles de usuarios concurrentes en pocos segundos, la persistencia centralizada se convierte en el cuello de botella del sistema, al acumular conexiones, bloqueos y latencia de disco, como se explica en (Richards & Ford, 2020). Este problema es especialmente evidente en una arquitectura de microservicios, donde cada servicio puede escalar de forma independiente, pero la capa de datos sigue compartiendo el mismo punto de contención y termina saturándose.

La **Space-Based Architecture (SBA)** propone una solución distinta: desacopla la lógica de negocio del almacén central y la mueve a memoria distribuida. En este modelo, cada **Processing Unit (PU)** encapsula tanto la lógica de la aplicación como un fragmento del estado en RAM. De esa forma, la consulta y la reserva inicial se ejecutan sin acceso directo a disco ni sincronización síncrona con la base de datos, tal como también describen Richards y Ford (Richards & Ford, 2020). La arquitectura se apoya en varios componentes clave:

- **Processing Units (Unidades de Procesamiento):** cada unidad mantiene una copia local del estado necesario para atender peticiones y resolver reservas con latencia mínima.
- **Virtualized Middleware:** distribuye la carga entre PUs y coordina el estado compartido mediante un *Messaging Grid*, un *Data Grid* y un *Processing Grid*. Este middleware permite equilibrar tráfico y crear nuevas instancias según la demanda.
- **Escritura asíncrona:** las transacciones confirmadas en memoria se registran de forma desacoplada y se persisten en segundo plano, evitando que el usuario espere la latencia real de la base de datos, como se muestra en (Richards & Ford, 2020).

En conjunto, la SBA ofrece una elasticidad mucho mayor para picos repentinos de tráfico y mantiene tiempos de respuesta muy bajos, incluso cuando la demanda crece de forma explosiva.

2.2. Colisiones de datos en asientos numerados

La ventaja de rendimiento de la SBA no es gratuita: al replicar el estado en memoria para escalar, surge el riesgo de **colisiones de datos** y condiciones de carrera cuando dos usuarios intentan reservar el mismo asiento simultáneamente, como también lo señalan Richards y Ford (Richards & Ford, 2020). Para evitar reservas dobles o *double-booking*, la arquitectura combina varios mecanismos de control.

1. **Particionamiento del data grid:** el inventario de asientos se divide por particiones, y cada una tiene un nodo primario responsable de la verdad para ese subconjunto. El *Messaging Grid* redirige cada solicitud a la partición adecuada, de modo que el mismo asiento siempre se procesa por la misma unidad y de forma secuencial.
2. **Bloqueo temporal en memoria con TTL:** cuando un usuario selecciona un asiento, la PU lo marca como `RESERVADO_TEMPORAL` durante un intervalo breve. Mientras ese bloqueo esté activo, cualquier intento concurrente de reservar el mismo asiento se rechaza inmediatamente sin consultar la base de datos.
3. **Trade-offs del diseño:**
   - **Consistencia frente a disponibilidad durante failover:** si el nodo primario falla, el respaldo asume la primacía en una pequeña ventana temporal. En ese intervalo se sacrifica disponibilidad momentánea para preservar la consistencia del estado.
   - **Riesgo en la persistencia asíncrona:** la escritura diferida exige una capa de registro fiable para evitar pérdidas de reservas confirmadas si el clúster de memoria cae antes de sincronizar los datos con disco.

En otras palabras, la SBA es muy eficaz para manejar picos extremos de carga, pero exige una coordinación cuidadosa para equilibrar rapidez, consistencia y tolerancia a fallos.

3. Pregunta 3: Fallos en una arquitectura dirigida por eventos

3.1. Impacto del fallo

En una topología de Broker no existe un coordinador central que supervise el flujo. Cada procesador de eventos consume eventos de una cola y, si los procesa con éxito, publica nuevos eventos que otros consumidores recogerán. Cuando un procesador falla de forma repetida ante un tipo concreto de mensaje, se produce una cadena de consecuencias que afecta tanto al flujo del sistema como a la experiencia del usuario final (Richards & Ford, 2020).

El primer efecto es que el evento que provocó el fallo queda atascado. Si el broker utiliza un modo de acknowledgment automático, el mensaje se marca como entregado antes de procesarse y se pierde definitivamente; si utiliza acknowledgment manual, el mensaje permanece retenido por el procesador defectuoso, impidiendo que otros consumidores lo recojan. En ambos casos, los eventos posteriores que dependían de él nunca se generan, y el flujo de negocio se detiene en ese punto.

Para el usuario final el impacto es una ausencia total de retroalimentación. Como la comunicación es asíncrona y no hay un mediador que centralice el estado del workflow, el usuario no recibe confirmación ni mensaje de error: simplemente observa que la operación no progresa. Desde el punto de vista operativo, la topología de Broker tampoco ofrece visibilidad global del flujo, de modo que diagnosticar la causa raíz requiere revisar logs dispersos en varios procesadores (Ford et al., 2021). El fallo de un solo componente se traduce en una cadena silenciosa de eventos no generados, sin que el sistema tenga un punto único donde detectarlo.

3.2. Patrón de eventos de flujo de trabajo

El Workflow Event Pattern es un patrón de arquitectura reactiva diseñado específicamente para gestionar fallos en flujos asíncronos sin sacrificar la capacidad de respuesta del sistema (Richards & Ford, 2020). No sustituye al Broker ni introduce un mediador de control de flujo: su único propósito es separar la gestión de errores del flujo normal.

La idea central es **delegar** el evento fallido a un componente especializado, el *workflow processor*, que se encarga de contener el error y orquestar la reparación. En lugar de que el procesador defectuoso reintente indefinidamente o descarte el mensaje, el workflow processor intercepta el evento, lo marca como fallido y lo redirige a un canal de reparación. Allí puede aplicar políticas de reintento con retroceso exponencial, mover el evento a una cola de mensajes muertos (*dead-letter queue*) o notificar a un operador humano a través de un *dashboard* para que corrija los datos. La figura 1 contrasta el flujo normal con el flujo de reparación.

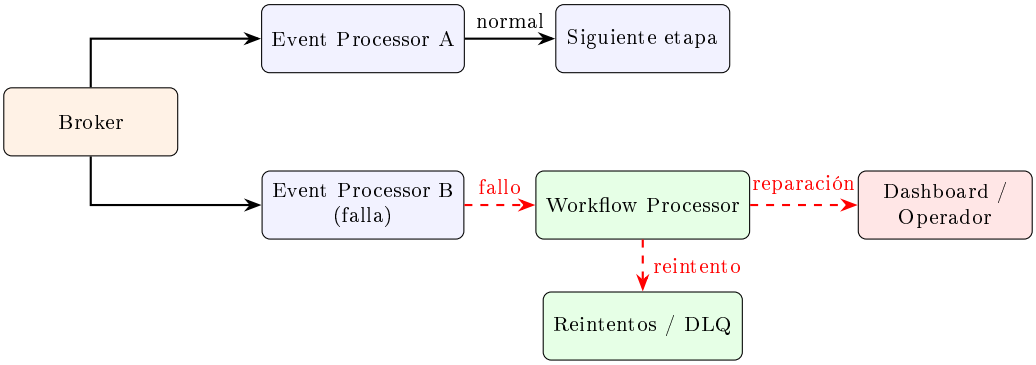


Figura 1: Workflow Event Pattern en una topología de Broker. El procesador B falla y delega el evento al Workflow Processor, que lo desvía a reparación manual (Dashboard) o a reintento/DLQ, sin bloquear el flujo normal del procesador A. Elaboración propia a partir de (Richards & Ford, 2020).

Los componentes adicionales que introduce el patrón son:

- **Workflow Processor:** recibe los eventos fallidos, mantiene el estado del flujo de reparación y decide la acción a tomar (reintentar, derivar a una cola de errores o escalar a un operador humano). No controla el flujo normal del negocio; solo actúa sobre los fallos (Richards & Ford, 2020).
- **Dashboard:** interfaz que muestra al operador los eventos atascados, el estado de los reintentos y las acciones disponibles. Permite la intervención manual cuando el error no puede resolverse automáticamente.
- **Cola de reparación (dead-letter queue):** canal donde se acumulan los eventos que no pudieron procesarse, para su análisis posterior o su reprocesamiento una vez corregidos los datos.

El patrón mitiga el problema porque convierte un fallo catastrófico en un incidente contenido: el evento defectuoso sale del flujo normal y entra en un circuito de reparación con visibilidad y estado propio, sin arrastrar al resto de la cadena. El sistema sigue procesando los eventos posteriores que no dependen del fallido, y el operador puede actuar sobre el evento atascado cuando lo considere.

Esta mitigación tiene un coste, expresado en los siguientes trade-offs:

- **Procesamiento fuera de orden:** como los eventos fallidos se reintentan o reparan en un momento posterior, pueden llegar a los consumidores después de eventos que originalmente les seguían. Esto puede romper invariantes de negocio que asumían un orden secuencial (Ford et al., 2021). Para mitigarlo se pueden usar identificadores de correlación, números de versión o un buffer de reordenación.
- **Mayor complejidad operativa:** aparecen nuevos componentes (workflow processor, dashboard, cola de reparación) que hay que desplegar, monitorizar y mantener. La topología de Broker deja de ser simple y gana un subsistema de gestión de errores.
- **Posible duplicación de eventos:** si el reintento se produce después de que el procesador original ya hubiera realizado parte del trabajo (por ejemplo, un efecto secundario), el reprocesamiento puede duplicar la acción. El diseño debe ser idempotente o incluir deduplicación (Ford et al., 2021).
- **Latencia en la resolución:** la reparación manual o los reintentos con retroceso exponencial introducen un retraso entre el fallo y la resolución final, lo que puede ser inaceptable para flujos con requisitos de tiempo real.

A pesar de estos trade-offs, el patrón es preferible a las alternativas en la topología de Broker. Sin él, un solo procesador defectuoso puede detener una cadena entera de eventos sin que el sistema tenga visibilidad ni capacidad de recuperación. El Workflow Event Pattern transforma ese fallo silencioso en un incidente observable y gestionable, preservando la escalabilidad y el desacoplamiento propios del Broker.

4. Pregunta 4: Granularidad de los microservicios de recursos humanos

4.1. Error de fondo en el diseño

El equipo creó un servicio por cada entidad del monolito: Empleado, Departamento, Nómina y Beneficios. Ese corte sigue las tablas, no un bounded context. Un contexto delimitado agrupa un modelo y un lenguaje de negocio que se mantienen consistentes dentro de una frontera. No coincide con cada sustantivo del esquema relacional (Richards & Ford, 2020). Contratar a una persona es una sola capacidad. Al partirla en cuatro servicios, la cohesión de esa operación se convirtió en una cadena de llamadas síncronas.

El capítulo 17 trata la granularidad como el equilibrio entre fuerzas que justifican partir un servicio y fuerzas que justifican dejarlo junto. El propósito, o alcance funcional, empuja a separar lo que cambia por razones de negocio distintas. Las transacciones y la coreografía empujan en sentido contrario: si una operación tiene que confirmarse como una unidad, o si completarla exige un ida y vuelta constante entre servicios, esos servicios quedaron demasiado finos (Richards & Ford, 2020). El equipo aplicó un desintegrador mal leído (un servicio por entidad) y no miró los integradores. Contratar no puede terminar si falla cualquiera de los cuatro saltos, y cada salto añade connascencia de tiempo. El resultado es un monolito distribuido: el acoplamiento del sistema original, ahora pagado en red, latencia y fallos parciales (Ford et al., 2021).

4.2. Nueva descomposición

La frontera nueva sale de cruzar propósito, transacciones y coreografía sobre la operación de contratación, no de contar entidades.

El propósito pide un servicio por capacidad y por contexto. En este dominio hay al menos dos lenguajes. El de la relación laboral dice quién es la persona, a qué departamento entra y en qué fecha empieza. El de la compensación dice qué se le paga y qué beneficios recibe. Nómina y beneficios cambian por reglas de pago y por la confidencialidad de esos datos. El alta cambia por la estructura organizativa. Separar esos dos lenguajes tiene un propósito. Separar Empleado de Departamento no lo tiene: durante la contratación, ninguno de los dos significa nada sin el otro.

Las transacciones piden que lo que debe confirmarse junto viva en el mismo servicio, donde cabe una transacción ACID local. Crear el empleado y asignarlo a un departamento es una sola unidad de trabajo. No debe quedar un alta confirmada sin departamento, ni una plaza reservada si el alta se revierte. Eso une Empleado y Departamento en un servicio de Personal. Abrir la nómina puede ocurrir un poco después sin romper una invariante que el usuario espere en la misma petición. Esa operación no obliga a compartir la transacción ni la base de datos.

La coreografía pide mirar el caso de uso completo. Si contratar exige llamadas síncronas a casi todos los servicios, los límites están mal puestos. O se fusionan los participantes de esa cadena, o el cruce entre contextos deja de ser síncrono. La contratación queda así. El servicio de Personal ejecuta el alta en una transacción local. Al confirmar, publica `EmpleadoContratado`. El servicio de Compensación, que reúne Nómina y Beneficios, consume el evento y abre salario y beneficios en su propia transacción. La figura 2 muestra el cambio: de cuatro saltos síncronos a un evento en la frontera de los dos contextos.

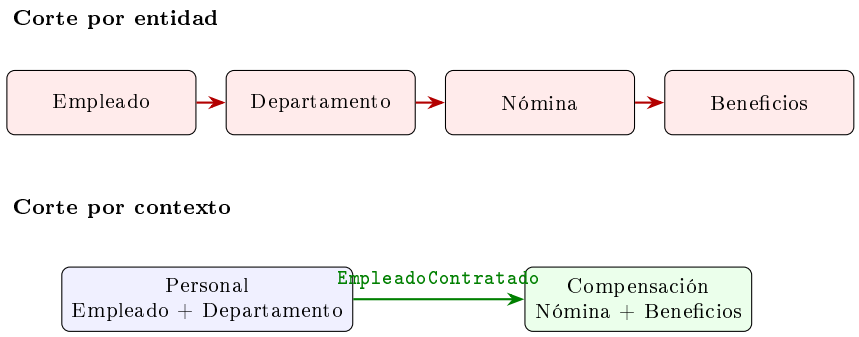


Figura 2: La contratación deja de ser una coreografía síncrona entre cuatro entidades y pasa a ser una transacción local en Personal seguida de un evento hacia Compensación. Elaboración propia a partir de (Richards & Ford, 2020).

El acoplamiento dinámico baja porque Personal no espera a Nómina ni conoce sus tablas. Un fallo de Beneficios no deshace el alta: el evento se reintenta. La consistencia entre contextos es eventual, y ese es el precio de no meter otra vez todo el monolito en un solo despliegue. Si el negocio exigiera que nadie quede contratado mientras la nómina no exista en el mismo instante, el integrador de transacciones mandaría unir ambos contextos o cubrir el hueco con una saga y sus compensaciones (Ford et al., 2021). Para la operación descrita, el evento en la frontera alcanza. Es el corte que saca la fragilidad del diseño original sin volver a un servicio por tabla.

5. Pregunta 5: Consistencia, despliegue y equipo en un sistema financiero

5.1. Consistencia de los datos

La Service-Based Architecture (SBA) organiza el sistema en un número reducido de servicios de dominio de grano grueso, desplegables de forma independiente pero apoyados en una base de datos compartida o parcialmente compartida (Richards & Ford, 2020). Esa base común es lo que permite que una operación que toca varios conceptos del negocio se resuelva dentro de una única transacción ACID: el servicio abre la transacción, modifica las tablas que necesita y confirma o revierte como una sola unidad. No hay coordinación distribuida porque no hay varias bases de datos que coordinar.

Los Microservicios siguen el principio opuesto: cada servicio posee su propia base de datos, derivada de un *bounded context* del dominio (Richards & Ford, 2020). Esa independencia de datos es la fuente de su agilidad, pero también la razón por la que la consistencia fuerte deja de ser gratuita. Cuando una operación cruza dos o más servicios, ya no existe una transacción única que la cubra. El arquitecto debe recurrir a sagas con compensaciones o, cuando el motor lo permite, a transacciones distribuidas, y aceptar que durante la ventana entre pasos el sistema se encuentra en un estado intermedio visible (Ford et al., 2021). En términos prácticos, los Microservicios ofrecen consistencia eventual entre contextos, no consistencia ACID global. La figura 3 contrasta ambos escenarios.

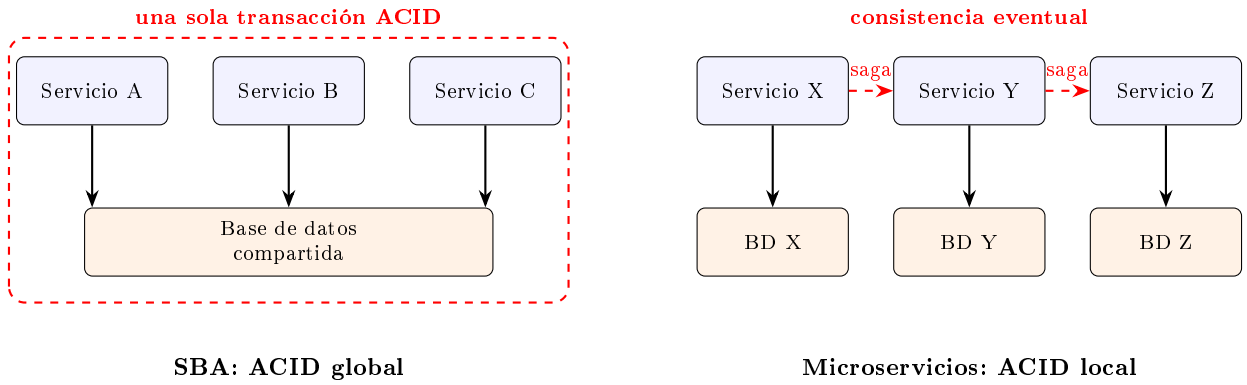


Figura 3: Consistencia en SBA y Microservicios. En SBA una única base de datos permite que una operación multi-concepto se resuelva como una transacción ACID. En Microservicios cada servicio confirma en su propia base de datos y la consistencia entre contextos se alcanza mediante sagas y compensaciones, con estados intermedios visibles. Elaboración propia a partir de (Ford et al., 2021; Richards & Ford, 2020).

Para el requisito de consistencia absoluta, la SBA es claramente más adecuada. No porque sea más moderna ni más elegante, sino porque la propiedad que se exige —atomicidad sobre operaciones que tocan varios conceptos del negocio— es trivial en un modelo de base de datos compartida y costosa de reconstruir en un modelo de datos por servicio.

5.2. Despliegue independiente

En SBA los servicios son desplegables de forma independiente en el sentido físico: cada uno es un artefacto separado que puede subir a producción sin recompilar los demás. Sin embargo, la base de datos compartida introduce un acoplamiento que la independencia de despliegue no elimina. Un cambio en el esquema afecta a todos los servicios que la usan, y coordinar ese cambio requiere planificar la migración con todos los equipos al mismo tiempo. La agilidad de despliegue existe, pero está limitada por el esquema compartido.

En Microservicios la independencia es más profunda: cada servicio controla su esquema, su ciclo de

vida y su versionado. Un equipo puede desplegar su servicio con un cambio de esquema sin coordinar con los demás, siempre que respete los contratos publicados. Esa es la fuente principal de su agilidad y, a la vez, la razón por la que requiere un equipo con madurez operativa: sin contratos claros, sin versionado de API y sin observabilidad distribuida, la independencia se convierte en fragilidad. La tabla 2 resume el contraste en los tres requisitos planteados.

Requisito	**SBA**	**Microservicios**
Consistencia ACID	Fuerte, dentro de una transacción sobre la BD compartida	Local por servicio; entre servicios, eventual con sagas
Despliegue independiente	Físicamente independiente, acoplado por el esquema de la BD	Independiente también a nivel de datos; requiere contratos y observabilidad
Equipo con experiencia mixta	Curva de aprendizaje moderada; el modelo es cercano al monolito	Exige madurez en DevOps, mensajería y trazabilidad distribuida

Tabla 2: Comparación de SBA y Microservicios frente a los tres requisitos del CTO. Elaboración propia a partir de (Richards & Ford, 2020).

Si se prioriza la consistencia sobre la agilidad de despliegue, el CTO debe aceptar un conjunto concreto de compensaciones. La primera es menor agilidad por cambio de esquema: cualquier modificación de la base compartida obliga a coordinar a todos los servicios que la usan (Richards & Ford, 2020). La segunda es mayor acoplamiento en el modelo de datos: los servicios comparten tablas y eso crea dependencias implícitas que no aparecen en los contratos de API, de modo que un cambio aparentemente local puede tener efectos en cascada. La tercera es granularidad más gruesa: para mantener la transacción dentro de una sola base de datos, los servicios deben agrupar conceptos que en un diseño de Microservicios estarían separados, lo que reduce el número de unidades desplegables y, con él, la capacidad de escalar componentes individuales. La cuarta es escalabilidad menos granular: en Microservicios se puede escalar solo el servicio que recibe la carga, mientras que en SBA el escalado suele hacerse por servicio de dominio completo.

A cambio, el CTO obtiene un sistema en el que las operaciones financieras críticas se resuelven con transacciones ACID nativas, sin sagas ni compensaciones, sin estados intermedios visibles y sin la complejidad operativa que exige un equipo con experiencia mixta. Este último punto es el que con menos frecuencia aparece en las comparaciones técnicas y, sin embargo, suele ser el que decide: Microservicios exige comodidad con despliegues automatizados, observabilidad distribuida y patrones de consistencia eventual, mientras que la SBA es más tolerante porque la cantidad de piezas distribuidas es menor y las herramientas necesarias son las mismas que un equipo con experiencia en aplicaciones monolíticas ya conoce. Para un sistema de procesamiento de transacciones financieras donde la corrección del dato es un requisito regulatorio y no una preferencia de diseño, ese intercambio es razonable.

6. Pregunta 6: Reutilización, de SOA a los microservicios

6.1. La reutilización como objetivo y como consecuencia

En las arquitecturas orientadas a servicios (SOA), la reutilización es uno de los principios que guían el diseño. La idea consiste en desarrollar servicios que representen capacidades de negocio y que puedan ser utilizados por diferentes aplicaciones o procesos de una organización. Por ejemplo, una empresa puede disponer de un servicio de Clientes que permita consultar y validar información, al que accedan los sistemas de ventas, facturación y soporte. De esta forma, una misma capacidad se implementa una vez y se ofrece mediante un contrato común (Richards & Ford, 2020).

En los microservicios, en cambio, la prioridad es mantener los servicios pequeños, autónomos y organizados alrededor de responsabilidades de negocio bien delimitadas. Cada equipo debe poder modificar y desplegar su servicio sin depender continuamente de otros equipos. Por esta razón, la reutilización no se considera un objetivo que deba perseguirse a cualquier precio. Si compartir una funcionalidad introduce dependencias que dificultan la evolución independiente de los servicios, puede resultar más conveniente mantener implementaciones separadas (Ford et al., 2021).

Esto no significa que los microservicios renuncien a reutilizar. La diferencia está en qué se comparte y cómo se hace. En lugar de construir un único componente que concentre las necesidades de varios dominios, se procura que cada servicio conserve la responsabilidad sobre su propia lógica. La siguiente tabla resume las principales diferencias entre ambos enfoques.

Dimensión	**SOA**	**Microservicios**
Unidad de reutilización	Servicios de negocio compartidos, que pueden ser utilizados por diferentes aplicaciones de la organización.	Capacidades de negocio delimitadas, expuestas mediante contratos y APIs independientes.
Mecanismo	Contratos de servicio comunes, integración mediante middleware y, en determinados casos, orquestación centralizada.	APIs, eventos, librerías específicas y servicios de infraestructura compartidos.
Organización de la lógica	Puede concentrarse en servicios empresariales compartidos.	Se distribuye entre servicios que mantienen la responsabilidad sobre su propio dominio.
Principal desafío	Evitar el acoplamiento excesivo a servicios comunes y a componentes centrales de integración.	Mantener la autonomía sin provocar duplicaciones innecesarias o divergencias entre servicios.

Tabla 3: Diferencias en el enfoque de reutilización entre SOA y microservicios. Elaboración propia a partir de (Ford et al., 2021; Richards & Ford, 2020).

6.2. Reutilización de lógica de negocio: la validación de clientes

La validación de clientes permite comprender mejor esta diferencia. En una arquitectura SOA tradicional, una organización podría implementar un servicio central de Clientes con operaciones para registrar, consultar y validar información. Los sistemas de ventas, facturación y soporte utilizarían ese mismo servicio, evitando implementar por separado las mismas reglas. La ventaja es que una modificación de esas reglas se realiza en un único lugar y puede beneficiar a todos los consumidores.

Sin embargo, esta solución también introduce dependencias. Si el servicio de Clientes deja de estar disponible, los sistemas que necesitan consultarlo pueden verse afectados. Además, un cambio en su contrato puede obligar a revisar a los diferentes consumidores. Cuando demasiadas aplicaciones dependen de un único servicio compartido, su evolución puede convertirse en un proceso lento y coordinado entre varios equipos.

En microservicios se intenta evitar que todos los contextos dependan de un único modelo de Cliente.

Por ejemplo, el servicio de Ventas puede necesitar información sobre el límite de crédito y la identificación fiscal, mientras que el de Envíos se interesa principalmente por la dirección de entrega. Aunque ambos trabajen con clientes, sus necesidades no son exactamente las mismas. Por ello, cada servicio puede mantener su propio modelo de datos y las reglas que corresponden a su responsabilidad. Se pueden compartir identificadores y determinados eventos, sin necesidad de utilizar un mismo modelo interno para toda la organización (Ford et al., 2021).

En este escenario, una regla de negocio propia de un contexto debe permanecer bajo la responsabilidad del servicio que conoce ese contexto. También puede recurrirse a una librería compartida cuando contiene una funcionalidad realmente común y estable. No obstante, cada servicio debe controlar la versión que utiliza, evitando que una actualización de la librería obligue automáticamente a modificar y desplegar todos los demás.

Hay casos en los que sí se justifica crear un servicio independiente. Por ejemplo, si se necesita comprobar la validez de una identificación fiscal consultando una autoridad externa, esa operación puede constituir una capacidad específica que varios servicios consuman mediante una API. Lo importante es que el servicio tenga una responsabilidad bien definida y no termine convirtiéndose en un lugar donde se concentre toda la información y las reglas relacionadas con los clientes.

Por tanto, la diferencia no consiste simplemente en duplicar o no duplicar código. Se trata de decidir dónde debe vivir cada regla de negocio y qué dependencias son razonables. Compartir una funcionalidad puede ahorrar trabajo, pero si obliga a varios servicios a evolucionar al mismo ritmo, puede terminar perjudicando la autonomía que se busca con los microservicios.

6.3. Reutilización operativa: logging y monitoreo

La reutilización de capacidades operativas presenta características diferentes, porque estas funciones no representan reglas propias de un dominio de negocio. El registro de actividad, la recopilación de métricas, la supervisión del rendimiento y la trazabilidad de las peticiones son necesidades comunes a prácticamente todos los servicios de un sistema.

En arquitecturas SOA, algunas de estas responsabilidades podían concentrarse en un bus de servicios empresarial (ESB), encargado de tareas como el enrutamiento, la transformación de mensajes, la seguridad y el registro de las comunicaciones. Este enfoque facilitaba la administración de la integración, pero también podía convertir al bus en un componente demasiado importante para el funcionamiento del sistema. Si todas las comunicaciones dependían de él, una avería o una saturación podía afectar a numerosos procesos.

En los microservicios, estas capacidades suelen proporcionarse mediante componentes de infraestructura y herramientas compartidas. Por ejemplo, los servicios pueden enviar sus registros a una plataforma centralizada, utilizar mecanismos comunes para recopilar métricas y disponer de trazabilidad distribuida para seguir una petición a través de varios componentes. También pueden emplearse agentes, sidecars o una malla de servicios para gestionar determinadas tareas de comunicación, seguridad y observabilidad.

De esta manera, cada servicio mantiene su lógica de negocio, mientras que la plataforma proporciona herramientas comunes para operarlo y supervisarlo. La centralización de los registros y las métricas no significa necesariamente que todas las peticiones deban pasar por un único componente. La recopilación puede realizarse de forma distribuida y los datos pueden analizarse posteriormente en sistemas compartidos.

Esta distinción resulta importante: compartir una herramienta de monitoreo no obliga a que los servicios tengan el mismo modelo de negocio ni que dependan de un único ciclo de despliegue. En cambio, compartir un servicio que concentra reglas de negocio utilizadas por numerosos contextos puede introducir dependencias mucho más difíciles de gestionar.

En definitiva, los microservicios no eliminan la reutilización operativa ni la reutilización de capacidades de negocio. Lo que buscan es evitar que estas últimas se conviertan en dependencias centrales que limiten la autonomía de los servicios. La infraestructura común facilita el funcionamiento del conjunto, mientras que la lógica de negocio permanece bajo la responsabilidad de los servicios que la necesitan.

6.4. Cuándo sigue justificándose la orquestación

La orquestación suele criticarse porque concentra la coordinación de varios servicios en un componente que conoce los pasos de un proceso y decide cuándo ejecutar cada uno. Esto puede introducir un punto de dependencia importante, aumentar la carga sobre el coordinador y reducir la autonomía de los participantes. Sin embargo, no siempre es la mejor alternativa distribuir toda la coordinación entre los propios servicios.

Conviene aclarar que la orquestación no es exclusiva de SOA ni la coreografía pertenece únicamente a los microservicios. Ambos estilos pueden utilizar cualquiera de estos mecanismos. La elección depende de las características del proceso que se quiere implementar.

En la coreografía, cada servicio reacciona a eventos publicados por otros componentes y ejecuta sus propias responsabilidades. No existe necesariamente un coordinador que conozca todo el proceso. Esto favorece la independencia de los participantes, aunque puede dificultar la comprensión del flujo completo cuando intervienen muchos servicios.

La orquestación, por su parte, establece un componente responsable de coordinar las operaciones. Este conoce el estado del proceso, determina qué acción corresponde ejecutar y puede gestionar los resultados, los tiempos de espera y las posibles recuperaciones. Aunque introduce una dependencia con el coordinador, también permite conocer con mayor claridad en qué punto se encuentra una operación.

La orquestación puede resultar conveniente en los siguientes escenarios:

- **Procesos largos con varias operaciones relacionadas.** Cuando una operación involucra diferentes servicios y no puede completarse mediante una única transacción de base de datos, es necesario controlar el estado del proceso y definir cómo actuar ante los fallos. Las sagas permiten gestionar estas operaciones mediante pasos y acciones de compensación.
- **Flujos con condiciones y diferentes caminos de ejecución.** Si el resultado de una operación determina cuál debe ser el siguiente paso, un coordinador puede mantener esa lógica en un lugar definido y facilitar su comprensión.
- **Procesos que requieren intervención humana.** En determinadas operaciones es necesario esperar la aprobación de un responsable, revisar una solicitud o solicitar información adicional. Un orquestador puede conservar el estado del proceso mientras espera esa intervención.
- **Necesidad de seguimiento y auditoría.** En sistemas donde es importante conocer qué operaciones se realizaron, cuándo ocurrieron y cuál fue su resultado, mantener un estado explícito del flujo facilita el seguimiento y la reconstrucción de los acontecimientos.
- **Integración con sistemas heredados o externos.** Algunos sistemas no permiten modificar su funcionamiento o solamente ofrecen interfaces síncronas. En esos casos, un motor de integración u orquestación puede facilitar la coordinación entre componentes con características diferentes.

La existencia de un coordinador no implica necesariamente que el sistema tenga un único punto de fallo. Si el proceso es importante, el orquestador puede diseñarse con persistencia de estado, mecanismos de recuperación y alta disponibilidad. También deben contemplarse los tiempos de espera, los reintentos controlados y la idempotencia de las operaciones, para evitar que una misma acción produzca resultados duplicados.

Por otro lado, no todos los procesos necesitan este nivel de coordinación. Cuando las operaciones son

independientes y basta con que cada servicio reaccione a determinados eventos, la coreografía puede ser una solución más sencilla. El problema aparece cuando un flujo complejo se distribuye entre demasiados participantes y resulta difícil conocer su estado o gestionar correctamente los fallos.

Por ello, una alternativa razonable en microservicios es utilizar orquestadores locales para procesos que necesitan coordinación explícita, sin convertirlos en componentes centrales encargados de controlar toda la organización. La decisión debe tomarse según las necesidades del proceso y no como una obligación impuesta por el estilo arquitectónico.

6.5. Un flujo que se beneficia de la orquestación explícita

Un ejemplo de proceso que puede beneficiarse de la orquestación es la reserva de un viaje que incluye la contratación de un vuelo, una habitación de hotel y el pago correspondiente. Aunque cada operación pertenece a un servicio diferente, el cliente espera recibir una única confirmación de su reserva.

Un posible flujo de trabajo sería el siguiente:

1. El servicio de Reservas registra la solicitud con estado `PENDIENTE` y solicita el inicio del proceso.
2. El sistema realiza una comprobación antifraude. Si la operación requiere una revisión adicional, el proceso queda en espera hasta recibir una decisión.
3. El servicio de Vuelos comprueba la disponibilidad y bloquea una plaza durante un tiempo determinado.
4. El servicio de Hoteles verifica la disponibilidad y bloquea una habitación bajo condiciones similares.
5. El servicio de Pagos solicita la autorización del importe correspondiente.
6. Si todas las operaciones necesarias se completan correctamente, se confirman las reservas y se registra la operación como completada.
7. Finalmente, se genera la factura y se envía una notificación al cliente con los detalles del viaje.

Un orquestador puede controlar el estado de cada etapa y decidir cómo continuar según el resultado recibido. Por ejemplo, si el pago es rechazado, debe solicitar la liberación de la plaza aérea y de la habitación que habían sido bloqueadas. Si la reserva del hotel falla después de haber bloqueado el vuelo, también será necesario liberar ese vuelo y comunicar al cliente que la operación no pudo completarse.

Otro caso puede producirse si el pago se autoriza, pero posteriormente falla la confirmación de una de las reservas. En ese escenario, el sistema debe ejecutar las acciones de recuperación correspondientes, como anular la autorización del pago y liberar los recursos reservados. Si el importe ya fue cobrado, podría ser necesario iniciar un reembolso.

Estas acciones no consisten necesariamente en deshacer de manera exacta cada operación técnica. Son acciones de compensación que buscan devolver el proceso a un estado aceptable desde el punto de vista del negocio. Por eso deben definirse teniendo en cuenta las características de cada servicio.

La siguiente figura representa de forma simplificada la coordinación de los principales participantes.

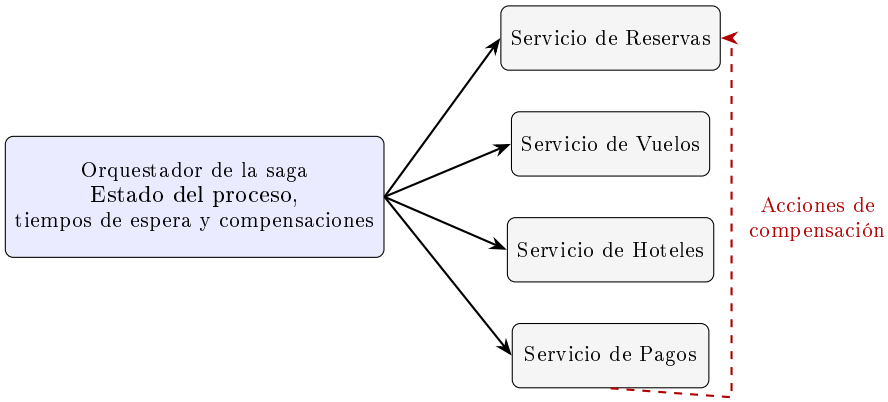


Figura 4: Representación simplificada de una saga orquestada para la reserva de un viaje. El orquestador mantiene el estado del proceso y coordina las operaciones de los servicios participantes. Elaboración propia.

La coreografía también podría utilizarse para implementar este proceso. Sin embargo, a medida que aumenta el número de participantes y aparecen condiciones, esperas y compensaciones, puede resultar más difícil conocer el estado global de la reserva. Cada servicio tendría que reaccionar correctamente a los eventos correspondientes y mantener su parte de la coordinación.

Con un orquestador, en cambio, existe un componente que conserva el estado general de la operación y conoce qué pasos se han completado y cuáles quedan pendientes. Esto facilita el seguimiento del proceso y la gestión de los errores, siempre que el coordinador esté diseñado para recuperarse ante fallos.

La siguiente tabla resume las diferencias entre ambos mecanismos de coordinación.

Criterio	**Coreografía**	**Orquestación**
Coordinación	Cada servicio reacciona a eventos y ejecuta su parte del proceso.	Un componente coordina las operaciones de los participantes.
Estado del flujo	Se encuentra distribuido entre los servicios.	Se mantiene de forma explícita en el coordinador.
Acoplamiento	Los participantes dependen de los eventos y contratos que intercambian.	Los participantes dependen de las instrucciones y contratos del orquestador.
Gestión de errores	Las reacciones y compensaciones deben coordinarse entre los participantes.	El coordinador puede gestionar los errores y decidir las acciones de recuperación.
Seguimiento	Puede requerir reconstruir el recorrido a partir de los eventos registrados.	Facilita consultar el estado y el historial del proceso.
Cuándo conviene	Flujos relativamente simples, con reacciones independientes entre servicios.	Procesos largos, con condiciones, esperas, compensaciones o intervención humana.

Tabla 4: Comparación entre coreografía y orquestación. Elaboración propia a partir de (Ford et al., 2021; Richards & Ford, 2020).

En conclusión, los microservicios no eliminan la reutilización, sino que cambian la manera de abordarla. La lógica de negocio debe permanecer bajo la responsabilidad del contexto que la necesita, mientras que las capacidades operativas pueden compartirse mediante una plataforma común. Por otra parte, la orquestación sigue siendo una herramienta válida cuando un proceso requiere coordinación explícita, seguimiento de su estado y gestión de compensaciones. El aspecto fundamental es utilizarla donde aporte valor, sin convertir un único componente en una dependencia obligatoria para el funcionamiento de todo el sistema.

Referencias bibliográficas

Ford, N., Richards, M., Sadalage, P., & Dehghani, Z. (2021). *Software Architecture: The Hard Parts: Modern Trade-Off Analyses for Distributed Architectures* (1st). O'Reilly Media.

Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach* (1st). O'Reilly Media.

Declaración de Uso de la Inteligencia Artificial

Durante la preparación de este documento se recurrió a distintas herramientas de inteligencia artificial: Claude (Anthropic), mediante el modelo Claude Opus 5.5; ChatGPT (OpenAI), mediante el modelo GPT 6 Luna; y Gemini (Google), en su versión más reciente, empleada a través de Gemini Notebook. Estas herramientas se utilizaron principalmente para resumir y facilitar la comprensión de la información contenida en la bibliografía recomendada, cuyas fuentes presentan una extensión considerable.

Los integrantes del equipo obtuvieron el contenido de este trabajo a partir del estudio de las fuentes bibliográficas recomendadas y se encargaron de redactarlo. Después, utilizaron la inteligencia artificial como apoyo para enriquecer los textos y perfeccionar su redacción.

Respecto a los diagramas, Olivia Ortiz Arboláez empleó Gemini Notebook para generar las imágenes correspondientes a los diagramas que tenía asignados, mientras que Anthony Cruz Garcia y Enrique González González utilizaron ChatGPT, en el modelo GPT 6 Luna, para elaborar los que les correspondían.

Por su parte, Jose Manuel Porras Guerra utilizó Claude (Anthropic), modelo Claude Opus 5.5, únicamente como herramienta de apoyo en la elaboración de la respuesta a la pregunta 5: le sirvió para localizar en la bibliografía orientada los pasajes pertinentes y sus páginas, y para contrastar posibles estructuras de la respuesta y un borrador inicial del texto, de las figuras y del código correspondiente. Dicho borrador fue únicamente un punto de partida: el integrante lo revisó, lo contrastó con las fuentes citadas y lo adaptó, de modo que el análisis y las decisiones finales son suyas.

Todo el contenido fue revisado, analizado y verificado conjuntamente por los integrantes del equipo, quienes asumen la responsabilidad final sobre el documento. Aunque se empleó la inteligencia artificial como herramienta de apoyo, los integrantes conocen y dominan los temas abordados en cada una de las preguntas.